



UNIVERSIDADE FEDERAL DO VALE DO SÃO FRANCISCO
CURSO DE GRADUAÇÃO EM ENGENHARIA ELÉTRICA

ENZO GONÇALVES SEVERO ROCHA
MARIA GABRIELA RODRIGUES GOMES
WESLEY SANTOS NUNES DE OLIVEIRA

PRÁTICA 2: TEMPORIZADORES

JUAZEIRO - BA

2026

UNIVERSIDADE FEDERAL DO VALE DO SÃO FRANCISCO
CURSO DE GRADUAÇÃO EM ENGENHARIA ELÉTRICA

ENZO GONÇALVES SEVERO ROCHA
MARIA GABRIELA RODRIGUES GOMES
WESLEY SANTOS NUNES DE OLIVEIRA

PRÁTICA 2: TEMPORIZADORES

Relatório apresentado à Universidade Federal do Vale do São Francisco – UNIVASF, Campus Tecnológico, como requisito para obtenção de nota na disciplina de Microcontroladores e Microprocessadores. Orientador: Prof. Dr. Ricardo Prates.

JUAZEIRO- BA

2026

SUMÁRIO

1. INTRODUÇÃO.....	3
2. REFERENCIAL TEÓRICO.....	4
2.1 CICLO DE MÁQUINA E TEMPORIZAÇÃO.....	4
2.2 TEMPORIZAÇÃO POR HARDWARE E PERIFÉRICO TIMER 0.....	4
3. METODOLOGIA.....	5
4. RESULTADOS E DISCUSSÕES.....	6
CONSIDERAÇÕES FINAIS.....	7

1. INTRODUÇÃO

A temporização é um dos pilares fundamentais no desenvolvimento de sistemas embarcados, sendo essencial para a sincronização de tarefas, controle de periféricos e a interação precisa com o ambiente externo. Em microcontroladores como o *PIC16F877A*, a gestão do tempo pode ser realizada através de duas abordagens principais: temporização por software e temporização por hardware.

A temporização por software baseia-se na execução de sequências de instruções que consomem ciclos de máquina propositalmente, como loops de repetição. Embora seja uma técnica de implementação simples, ela apresenta desvantagens significativas, como a ocupação integral da Unidade Central de Processamento (CPU) e a falta de precisão caso ocorram interrupções durante a contagem.

Em contrapartida, a temporização por hardware utiliza periféricos dedicados, especificamente o *Timer 0*. Este recurso permite que a contagem de tempo ocorra de forma independente à execução do programa principal, liberando a CPU para outras tarefas. Através do uso de *prescalers* e do sistema de interrupções, é possível alcançar uma precisão temporal elevada e uma eficiência computacional superior.

Esta atividade prática tem como objetivo a implementação de ambas as técnicas no *PIC16F877A*, utilizando o kit de desenvolvimento *ACEPIC PRO V2.1*. Através da alternância do estado de um LED em intervalos definidos, busca-se comparar o comportamento de cada método, consolidando o entendimento sobre registradores de controle (como *OPTION_REG* e *INTCON*), o cálculo de ciclos de máquina e a manipulação de flags de interrupção.

2. REFERENCIAL TEÓRICO

2.1 CICLO DE MÁQUINA E TEMPORIZAÇÃO

No microcontrolador *PIC16F877A*, a velocidade de execução das instruções está diretamente ligada à frequência do oscilador externo, F_{OSC} . Cada ciclo de máquina, T_{CM} , corresponde a 4 ciclos do oscilador:

$$T_{CM} = \frac{4}{F_{OSC}}$$

Para um cristal de 4 MHz, cada ciclo de máquina equivale a 1 μ s. A maioria das instruções em Assembly consome 1 ciclo de máquina, com exceção das instruções de desvio (como *GOTO* ou *BTFSS* quando a condição é verdadeira), que consomem 2 ciclos. O entendimento desta métrica é o que permite o cálculo preciso da temporização por software, onde o programador empilha instruções para atingir o tempo de atraso desejado.

2.2 TEMPORIZAÇÃO POR HARDWARE E PERIFÉRICO TIMER 0

O Timer 0 é um temporizador/contador de 8 bits que incrementa de 0 a 255 (00h a FFh). Ao passar de FFh para 00h, ocorre o fenômeno conhecido como *overflow* (estouro), que seta a flag *TOIF* no registrador *INTCON*.

Os principais componentes que regem o Timer 0 são:

- **Prescaler:** Um divisor de frequência programável (através do registrador *OPTION_REG*) que permite que o Timer 0 incremente a uma taxa menor que o ciclo de máquina (proporções de 1:2 até 1:256).
- **Registrador *TMR0*:** Pode ser pré-carregado com um valor inicial para ajustar o tempo exato até o estouro.
- **Interrupção:** O estouro do Timer 0 pode ser configurado para gerar uma interrupção automática, desviando o programa para o vetor 0004h.

3. METODOLOGIA

A metodologia adotada para esta prática consistiu na elaboração e análise de dois códigos em linguagem Assembly para o microcontrolador PIC16F877A. O objetivo central de ambos os programas foi realizar a alternância de estado de um LED conectado ao pino RD1 do port D, em intervalos definidos de aproximadamente 250 ms. Para demonstrar os diferentes paradigmas de projeto de sistemas embarcados, a prática foi dividida em duas abordagens de gestão de tempo: temporização por software e temporização por hardware. O desenvolvimento considerou a estrutura do kit ACEPIC PRO V2.1.

3.1. Parte 1: Temporização por Software

A primeira etapa explorou a criação de atrasos retendo o processamento da Unidade Central de Processamento (CPU) por meio de instruções ociosas e laços de repetição.

- **Configuração de I/O:** Inicialmente, o registrador **STATUS** foi manipulado para acessar o Banco 1 e configurar o registrador **TRISD**, definindo o pino RD1 como saída digital. Em seguida, retornou-se ao Banco 0 para inicializar o **PORTD** em nível lógico baixo (LED apagado).
- **Lógica Principal:** O loop principal do programa foi estruturado de forma estritamente sequencial. O código liga o LED, invoca a sub-rotina de atraso (`delay_250ms`), desliga o LED e invoca a sub-rotina novamente, repetindo esse ciclo infinitamente.
- **Construção do Atraso (Delay):** A rotina de atraso foi construída com dois laços de repetição aninhados, controlados pelas variáveis **CONT1** e **CONT2**, ambas inicializadas com o valor 250. O laço interno foi preenchido com 5 instruções **NOP**(No Operation), que consomem 1 ciclo de máquina cada. O acúmulo desses ciclos, somado ao tempo gasto pelas instruções de decremento condicional (`decsfz`) e salto (`goto`), gerou o atraso temporal necessário para a correta visualização do pisca-LED.

3.2. Parte 2: Temporização por Hardware (Timer 0)

A segunda etapa substituiu as sub-rotinas bloqueantes pelo uso do periférico **Timer 0** associado ao sistema de interrupções, permitindo que a contagem de tempo ocorresse em paralelo à execução do processador.

- **Configuração do Periférico Timer 0:** O registrador **OPTION_REG** foi configurado com o valor binário 0b11010011. Isso definiu o uso do clock interno de máquina e atribuiu um *prescaler* de 1:16 ao temporizador.
- **Parâmetros de Contagem:** O registrador **TMR0** foi pré-carregado com o valor 6, exigindo exatamente 250 incrementos de máquina até que ocorresse o estouro (overflow) ao passar de FFh (255) para 00h. Como um único estouro não fornecia a janela de tempo de 250 ms desejada, declarou-se uma

variável auxiliar chamada `CONT_OVF`, inicializada em 125, atuando como um multiplicador de estouros.

- **Configuração das Interrupções:** O registrador **INTCON** foi parametrizado para habilitar a chave geral de interrupções (bit GIE) e a permissão de interrupção específica do Timer 0 (bit T0IE).
- **Rotina de Serviço de Interrupção (ISR):** Alocada no vetor absoluto 0x0004, a ISR é acionada a cada estouro. Nela, o software verifica e limpa a flag de estouro T0IF, recarrega o TMR0 com 6 e decrementa o `CONT_OVF`. Apenas quando a variável `CONT_OVF` atinge zero (sinalizando a passagem completa do tempo estipulado), o estado do LED no pino RD1 é invertido utilizando a porta lógica XOR (`xorwf`). Em seguida, a variável é recarregada para o próximo ciclo.
- **Otimização do Loop Principal:** Diferentemente do primeiro método, o laço principal (loop) do código foi deixado intencionalmente vazio. Isso evidenciou o benefício da temporização por hardware, provando que a CPU pode ficar completamente livre para a execução de outras tarefas enquanto o periférico cuida da temporização de forma autônoma.

4. RESULTADOS E DISCUSSÕES

A implementação do sistema de acionamento periódico do LED (comutação a cada 250 ms, sob uma frequência de oscilador de 8 MHz) foi bem-sucedida em ambas as abordagens propostas. Considerando que o tempo de ciclo de máquina do microcontrolador utilizado é de 0,5 μ s, o intervalo de 250 ms demandou a contagem exata de 500.000 ciclos de instrução para a alternância de estado do pino RD1.

Na abordagem de temporização por software, o atraso foi obtido através de laços de repetição aninhados (loops) associados a instruções nulas (NOP). Observou-se que, embora o método atinja a precisão temporal exigida, ele impõe um comportamento de espera ocupada (busy waiting) ao microcontrolador. Durante o atraso, o fluxo de execução principal fica retido na rotina de contagem, bloqueando a Unidade Central de Processamento (CPU). Discute-se que esta estratégia apresenta baixíssima escalabilidade: a inserção de qualquer nova instrução para a leitura de sensores ou botões dentro do laço principal alteraria a quantidade de ciclos executados, descalibrando o tempo de 250 ms e exigindo um recálculo manual de toda a rotina.

Por outro lado, os resultados obtidos com a temporização por hardware, utilizando o módulo periférico Timer0 associado ao sistema de interrupções, demonstraram eficiência superior. Ao configurar o prescaler para 1:16 e iniciar a contagem do registrador TMR0 em 6, obteve-se um transbordamento (overflow) exato a cada 2 ms. O acúmulo de 125 transbordamentos na Rotina de Serviço de Interrupção (ISR) garantiu os 250 ms desejados. A análise arquitetural deste método evidencia que a contagem de tempo ocorre em segundo plano, delegada ao hardware do Timer0. Consequentemente, o laço de repetição principal (main loop) do código manteve-se completamente vazio e disponível.

A comparação entre as metodologias corrobora a literatura de sistemas embarcados: enquanto a temporização por software resulta em 100% de ociosidade produtiva da CPU durante o atraso, a técnica por hardware atua orientada a eventos. O processador é interrompido apenas por breves microssegundos para atualizar variáveis de controle e comutar o LED (via operação lógica XOR), retornando imediatamente à sua rotina principal. Isso viabiliza a execução de tarefas concorrentes e garante o determinismo do sistema.

CONSIDERAÇÕES FINAIS

O presente trabalho alcançou plenamente os objetivos propostos ao implementar e analisar duas estratégias distintas de controle temporal em um microcontrolador da família PIC. O requisito funcional de gerar um sinal intermitente (pisca-pisca) com período de 0,5 segundos (0,25 s em nível alto e 0,25 s em nível baixo) foi rigorosamente atendido tanto pelo método de atraso por código (software) quanto pelo método baseado em periférico (hardware).

A análise comparativa entre as abordagens permitiu compreender, na prática, os impactos das escolhas de projeto no desenvolvimento de firmware. A temporização por software, baseada em laços de repetição, provou-se um método simples e de grande valor pedagógico para a compreensão dos ciclos de máquina. Contudo, sua natureza inviabiliza sua aplicação em projetos complexos, visto que impede o microcontrolador de reagir a outros estímulos externos de forma simultânea.

Em contrapartida, a implementação através do Timer0 com interrupções demonstrou a superioridade técnica da temporização por hardware. Este método otimiza drasticamente os recursos de processamento, conferindo alta escalabilidade e modularidade ao código. Ao liberar o processador principal das rotinas de contagem, o sistema torna-se capaz de realizar múltiplas tarefas de forma assíncrona.

Conclui-se, portanto, que para o projeto de sistemas embarcados robustos, responsivos e profissionais, a delegação de tarefas repetitivas (como a medição de tempo) aos periféricos de hardware dedicados é uma prática essencial e efetiva.